

CloudShield AI

09 — Intelligence & AI Architecture Specification

Version 1.0 | Product-Level Engineering Specification

CloudShield AI / CloudSpartanX

Document purpose: Define the architecture, contracts, reasoning flow, model abstraction, context engineering, safety boundaries, evaluation strategy, and operational design of the CloudShield AI intelligence layer.

Canonical principle: AI proposes. Deterministic systems validate. Governance decides. Cloud APIs execute. Evidence records. Trusted state is earned.

1. Purpose and Scope

This specification defines how intelligence is produced and consumed across CloudShield AI. It covers the AI architecture as a product-wide capability spanning security analysis, risk explanation, dependency-aware reasoning, remediation assistance, operational intelligence, and decision support.

- Define the boundary between deterministic product logic and probabilistic AI reasoning.
- Define a provider-neutral AI abstraction layer so hosted and future local/open-weight models can be exchanged without redesigning the product.
- Define context assembly, retrieval, prompting, structured outputs, validation, model routing, and observability.
- Define how AI interacts with CloudShield AI domain objects and the DAIL subsystem.
- Establish safety, security, privacy, reliability, and evaluation requirements for AI features.
- Provide an implementation-ready architecture for the MVP without requiring training a foundation model from scratch.

Non-goals: training a proprietary foundation model for MVP, allowing an LLM to directly execute arbitrary cloud commands, replacing deterministic security rules with model judgment, or making AI output equivalent to verified/trusted infrastructure state.

2. Architectural Position of AI

AI is an intelligence subsystem within CloudShield AI, not the product itself. The product combines deterministic cloud inventory, security analysis, risk, dependency and impact analysis, lifecycle controls, governance, evidence, and AI-assisted reasoning.

Layer	Responsibility	AI Role
Product	User workflows, findings, remediation, dashboards	Consumes intelligence; does not delegate authority to the model.
Platform	Identity, tenancy, jobs, data, policy	Provides controlled context and authorization.
Intelligence	Reasoning, explanation, recommendation, summarization	Primary AI execution boundary.
Security	Rules, policy, risk, controls	Provides deterministic facts and gates.

Layer	Responsibility	AI Role
Infrastructure	AWS/cloud integrations, execution	AI has no unrestricted direct authority.
DAIL subsystem	Candidate lifecycle, impact, verification, promotion	Consumes validated AI-assisted candidates; enforces deterministic lifecycle safety.

3. AI Architectural Principles

- Provider neutrality: model vendors are adapters, not architectural dependencies.
- Context before prompt: model quality depends on curated, authorized, versioned context rather than raw data dumps.
- Structured output over free-form authority: machine-consumable AI responses use explicit schemas.
- Deterministic validation after generation: model output is advisory until independently validated.
- Least context: provide only the minimum tenant-scoped information required for the task.
- Provenance: facts used in important recommendations must be traceable to observations, findings, policies, dependencies, or artifacts.
- Reproducibility: record model/provider/version, prompt/template version, context references, and output schema version for material AI decisions.
- Human governance for consequential actions.
- Fail closed when required deterministic evidence or validation is unavailable.
- Evaluation is a product capability, not a one-time model-selection exercise.

4. Intelligence Architecture

4.1 Canonical architecture

The recommended MVP architecture separates AI orchestration from domain services and model providers:

Component	Responsibility	Authority
AI Gateway	Authenticated entry point for AI jobs/requests; quotas and routing	No cloud mutation authority
AI Orchestrator	Select task, assemble context, invoke model, validate response, persist trace	No direct privileged cloud access
Context Builder	Resolve authorized facts, snapshots, findings, dependencies, policies, candidate versions	Read-only domain access
Prompt/Policy Registry	Versioned task instructions, schemas, safety rules, model configuration	Configuration authority only
Model Adapter	Provider-specific request/response translation	Provider API access only
Output Validator	Schema, domain, safety, policy, and consistency checks	Can reject; cannot self-authorize promotion
Tool Broker	Optional controlled tool execution for future workflows	Independent authorization required
AI Evaluation Service	Offline/online evaluation, regression, benchmark tracking	No production mutation authority
Evidence Recorder	Persist request metadata,	Append/evidence authority only

Component	Responsibility	Authority
	references, validation outcomes, material outputs	

4.2 Logical flow

1. A product workflow requests an intelligence task.
2. The AI Orchestrator authenticates the request and determines tenant/scope from trusted context.
3. The Context Builder retrieves only authorized and relevant domain facts.
4. The task policy selects an approved prompt/template, output schema, model route, and safety profile.
5. The model adapter invokes the selected provider/model.
6. The raw response is treated as untrusted data.
7. The Output Validator parses and validates the response against schema and deterministic domain rules.
8. Validated intelligence is stored with provenance and linked to the source context/version.
9. If the output proposes infrastructure changes, it becomes an explicit RemediationCandidate and enters the DAIL lifecycle.
10. The product presents the result with confidence, evidence, limitations, and appropriate governance controls.

5. AI Task Taxonomy

Task ID	Task	Inputs	Output	Authority Level
AI-T01	Finding explanation	Finding, rule, observations, evidence	Structured explanation	Advisory
AI-T02	Risk reasoning	Finding, asset, exposure, dependencies, controls	Risk rationale/priorities	Advisory
AI-T03	Dependency reasoning	Graph context, resource metadata, change	Relationship hypothesis	Advisory; must be validated
AI-T04	Impact explanation	Candidate, dependency graph, policy	Impact narrative and unknowns	Advisory
AI-T05	Remediation recommendation	Finding, current config, constraints, policy	Structured recommendation	Candidate-generating
AI-T06	Infrastructure change proposal	Authorized candidate context	Machine-readable proposed change	Candidate-generating; never trusted directly
AI-T07	Operational summary	Jobs, findings, changes, drift	Summary	Advisory
AI-T08	Investigation assistant	Authorized evidence/context	Questions, hypotheses, next checks	Advisory
AI-T09	Report drafting	Validated domain facts	Draft report text	Advisory
AI-T10	Policy assistance	Policy text + validated facts	Suggested policy interpretation	Advisory; deterministic policy remains authoritative

6. Model Abstraction Layer

6.1 Requirement

CloudShield AI shall not couple domain logic directly to a specific LLM SDK or provider. All model interaction passes through a stable internal interface.

Interface Concept	Required Data
ModelRequest	Task ID, model route, system/task instructions, structured context, output schema, safety profile, correlation ID.
ModelResponse	Structured output or raw provider response, usage metadata, provider/model identifiers, finish/status information.
ModelError	Provider error class, retryability, safe diagnostic information, correlation ID.
ModelCapabilities	Context limits, structured-output support, tool support, modalities, latency/cost characteristics.
ModelPolicy	Allowed tasks, data classes, regions/provider constraints, retention rules, budget and rate limits.

6.2 Provider architecture

MVP should support one primary hosted model provider behind the abstraction layer. A future LocalModelAdapter can implement the same contract for an on-device, private, or self-hosted open-weight model.

Adapter	Purpose	MVP
HostedLLMAdapter	Connect to selected external model provider	Required
LocalLLMAdapter	Connect to future self-hosted/open-weight model	Future
MockLLMAdapter	Deterministic test responses	Required
EvaluationLLMAdapter	Controlled benchmark/replay execution	Recommended

7. Context Engineering

7.1 Context hierarchy

AI context should be assembled from canonical domain objects rather than raw unrestricted database or cloud access.

Context Tier	Examples	Handling
Identity	Tenant, account, region, user role, authorization scope	Mandatory scope boundary; never model-authoritative.
Observed facts	Assets, observations, snapshots, cloud metadata	Read-only facts with timestamps/provenance.
Security facts	Findings, rule IDs, control status, risk assessments	Deterministic results preferred.
Relationship facts	Dependency edges, impact results, confidence, provenance	Include freshness and unknowns.
Policy	Applicable security/change policies and constraints	Clearly labeled authoritative policy.

Context Tier	Examples	Handling
Candidate context	Exact candidate/version, diff, intended targets	Immutable reference for reasoning.
Evidence	Verification artifacts, logs, test results	Include references rather than unnecessary raw secrets.
User request	Question or objective	Treat as potentially untrusted input; cannot override system policy.

7.2 Context controls

- Apply tenant and authorization filtering before context assembly.
- Prefer compact canonical summaries to uncontrolled raw payloads.
- Attach source identifiers and timestamps to material facts.
- Clearly distinguish instructions, policy, and untrusted retrieved content.
- Redact secrets and unnecessary personal/customer information.
- Limit context size using deterministic relevance rules before model invocation.
- Version context recipes so material AI results can be reproduced or audited.
- Do not allow retrieved content to silently become higher-priority instructions.

8. Retrieval and Knowledge Architecture

CloudShield AI should initially rely primarily on its canonical domain model, structured queries, security rules, dependency graph, and evidence store. A vector/semantic retrieval layer may be added where unstructured knowledge materially improves the product.

Knowledge Source	Primary Retrieval Method	Reason
Assets/configurations	Structured queries	Exactness and freshness.
Findings/rules	Structured queries	Deterministic traceability.
Dependencies/impact	Graph/relational queries	Relationship correctness.
Policies	Versioned structured/text retrieval	Governance authority.
Evidence	Metadata + targeted artifact retrieval	Forensic provenance.
Documentation/runbooks	Semantic retrieval when needed	Useful for explanations and operational guidance.

A retrieval result is evidence/context, not authority. Retrieved text must remain labeled as data and subject to the same authorization and prompt-injection protections as other untrusted content.

9. Prompt and Instruction Architecture

9.1 Prompt layers

Layer	Purpose	Authority
System safety instructions	Model behavior and output constraints	Highest model-level instruction
Task instructions	Define requested reasoning task	Controlled by application
Policy context	Business/security constraints	Authoritative domain input; deterministic policy still wins
Structured facts	Cloud and product context	Data only
Untrusted content	Resource descriptions, logs,	Never treated as trusted

Layer	Purpose	Authority
	documents, user text	instructions
Output schema	Machine-readable contract	Enforced outside the model

9.2 Prompt requirements

- Prompt templates are version-controlled and reviewed.
- Every production task has an explicit task ID and expected output schema.
- Prompts must not contain raw credentials or unnecessary secrets.
- Untrusted retrieved content is clearly delimited and labeled.
- The model is instructed to distinguish known facts, inference, uncertainty, and missing information.
- Prompt changes affecting material decisions require evaluation before production rollout.
- Prompt templates must not be the only safety control; deterministic validators remain mandatory.

10. Structured Output Contracts

AI outputs used by product workflows must use versioned schemas. Free-form text may be displayed for explanation but cannot serve as an authoritative machine-action contract.

Output Field	Requirement
task_id	Matches the requested registered task.
schema_version	Known supported schema version.
summary	Concise explanation; not an authority signal.
facts	Claims traceable to supplied context where possible.
inferences	Clearly separated from observed facts.
uncertainties	Explicit missing/ambiguous information.
recommendations	Structured suggestions with rationale.
proposed_changes	Only when task permits; must reference stable asset IDs and exact target scope.
risk_notes	Potential negative outcomes and assumptions.
evidence_refs	References to canonical observations/findings/artifacts.
confidence	Calibrated/defined confidence representation; not proof of correctness.

11. Output Validation Pipeline

11. Parse response against the expected schema.
12. Reject malformed, truncated, or unexpected fields where strictness is required.
13. Validate identifiers against the canonical domain model.
14. Validate tenant/scope consistency.
15. Validate referenced resources and versions.
16. Validate policy constraints and prohibited operations.
17. Check for unsupported assumptions and missing mandatory evidence.
18. Normalize structured values into canonical forms.
19. Create a domain object only after successful validation.
20. Route infrastructure-affecting proposals into the RemediationCandidate/DAIL lifecycle.

A model saying that a change is safe is never sufficient evidence that the change is safe.

12. Tool Use and Agentic Boundaries

12.1 MVP stance

MVP should minimize agentic behavior. Read-only information retrieval can be brokered where useful; infrastructure mutation must remain outside direct model control.

12.2 Tool broker

Control	Requirement
Tool allowlist	Only registered tools can be requested.
Argument schema	Arguments validated before execution.
Authorization	Independent server-side authorization; never delegated to model.
Scope	Tenant/account/region/resource scope enforced by broker.
Risk class	Each tool has an explicit risk classification.
Confirmation	High-impact operations require human/policy confirmation.
Audit	Tool request, decision, execution, and result are recorded.
Rate/timeout	Bound execution cost and duration.
Result sanitization	Tool results are filtered before returning to model.

The model may request a tool, but the broker decides whether the tool may run.

13. AI and DAIL Integration

AI and DAIL are deliberately separated. AI can generate or refine a RemediationCandidate, while DAIL determines whether that candidate can advance through deterministic lifecycle gates.

Stage	AI Role	DAIL Role
Discovery	Optional summarization	None/indirect
Analysis	Explain and reason	Consumes deterministic facts
Recommendation	Generate candidate suggestion	None until candidate exists
Candidate creation	Provide structured proposal	Assign identity/version and lifecycle state
Dependency/impact	Explain or hypothesize	Consume canonical dependency/impact results
Verification	Interpret results	Own verification gates and exact-version binding
Approval	Provide rationale	Governance state remains external to model
Promotion	No authority	Own promotion controls
Trusted state	No authority	Trusted state earned after successful governed promotion

14. AI Safety Architecture

- Treat all model output as untrusted until validated.
- Prevent prompt injection from changing application policy or authorization.
- Prevent indirect prompt injection from cloud resource text, logs, tags, documents, or external integrations.

- Never place unrestricted credentials in model context.
- Separate advisory responses from candidate-generating responses.
- Require deterministic checks for security claims and infrastructure changes.
- Use policy-enforced tool brokerage for any future tool-enabled capability.
- Record material AI decisions and their context references.
- Support model/provider fallback without silently weakening safety controls.
- Disable or degrade safely when the model provider is unavailable.

15. AI Security and Privacy Controls

Threat	Architecture Control
Prompt injection	Instruction/data separation, task templates, validation, adversarial evaluation.
Indirect injection	Untrusted-content labeling, retrieval sanitization, deterministic policy checks.
Data exfiltration	Tenant-scoped retrieval, minimization, redaction, output controls.
Hallucination	Canonical ID validation, evidence references, deterministic checks.
Model overreach	No direct cloud authority; tool broker; DAIL gates.
Provider data retention	Provider policy controls, minimization, configurable retention.
Cross-tenant leakage	Context authorization before model invocation; tenant-aware cache keys.
Prompt/model manipulation	Versioned templates, model allowlists, evaluation gates.
Sensitive logs	Redaction and structured telemetry.

16. AI Reliability and Failure Handling

Failure	Required Behavior
Model timeout	Bounded retry if safe; otherwise explicit AI_FAILED state.
Provider outage	Route to approved fallback or return deterministic non-AI result.
Malformed output	Reject and record validation failure.
Low-confidence output	Present uncertainty; do not automatically escalate authority.
Context unavailable	Do not fabricate missing facts; fail or return limited deterministic result.
Policy unavailable	Block safety-critical candidate/promotion workflow.
Tool failure	Return explicit tool failure; do not infer success.
Rate limit	Backoff/queue; preserve user/job state.
Conflicting model outputs	Treat as uncertainty; use deterministic evidence or human review.

17. Model Routing and Selection

Model selection should be task-driven rather than hard-coded globally. A routing policy can choose among approved models based on capability, context size, latency, cost, privacy constraints, and task criticality.

Routing Factor	Rule
Task capability	Model must support required structured-output/context features.
Data sensitivity	Sensitive contexts only use providers/configurations approved for that data class.
Risk level	High-impact tasks use approved high-reliability model routes and stronger validation.
Latency	Interactive tasks favor low-latency routes within quality thresholds.
Cost	Batch/summarization tasks may use lower-cost approved models.
Availability	Fallback only to routes with equivalent safety controls.
Version	Material model upgrades require evaluation before automatic adoption.

18. Evaluation Architecture

18.1 Evaluation dimensions

- Correctness against known facts.
- Groundedness and citation/provenance quality.
- Security-rule consistency.
- Remediation validity.
- False-positive and false-negative behavior where measurable.
- Prompt-injection resistance.
- Tool-use safety.
- Tenant isolation.
- Latency and cost.
- Stability across model/provider versions.

18.2 Evaluation sets

Set	Purpose
Golden cases	Known expected answers and decisions.
Adversarial cases	Prompt injection, malicious resource text, unsafe requests.
Cloud scenarios	Representative AWS configurations and security findings.
Change scenarios	Candidate, dependency, impact, verification, drift cases.
Regression set	Previously failed production/test cases.
Human-reviewed set	Expert assessment of usefulness, clarity, and actionability.

Evaluation artifacts should be versioned and tied to model, prompt, schema, and context versions. Model upgrades should not be treated as safe solely because general benchmark scores improve.

19. Observability and AI Telemetry

Telemetry	Purpose	Sensitive Data Rule
-----------	---------	---------------------

Telemetry	Purpose	Sensitive Data Rule
Request metadata	Latency, task, route, status	Do not log raw secrets.
Model metadata	Provider/model/version, usage	Record identifiers, not unnecessary payloads.
Context metadata	Sources, versions, counts	Prefer references and hashes over raw sensitive data.
Validation result	Accepted/rejected and reason class	Safe structured diagnostics.
Safety events	Injection/tool/policy violations	Preserve forensic signal without secret leakage.
Quality metrics	Groundedness, validity, user feedback	Aggregate where possible.
Cost metrics	Token/usage spend by task/tenant	Tenant-aware access controls.

20. Data Storage and Retention

- Persist material AI requests/results only when needed for product functionality, evaluation, audit, or reproducibility.
- Store large prompts/artifacts in controlled object storage rather than unnecessarily expanding transactional records.
- Use references and hashes to bind AI outputs to context without duplicating all sensitive source data.
- Apply tenant-scoped authorization to AI traces and evaluation artifacts.
- Define retention classes for production traces, safety events, and benchmark data.
- Support deletion/expiration policies without breaking immutable security evidence requirements.

21. Performance and Scalability

Workload	Architecture Strategy
Interactive explanation	Synchronous bounded request; cache safe deterministic context; strict timeout.
Heavy investigation	Asynchronous job with progress state.
Large report generation	Background worker and artifact storage.
Batch AI evaluation	Dedicated worker pool with quotas.
High-volume findings	Aggregate deterministic facts before model invocation.
Provider throttling	Queue, backoff, per-tenant/provider quotas.
Model cost growth	Task-specific routing, caching, context compression, budgets.

The system should avoid invoking the model when deterministic rules can answer the question completely and safely.

22. Caching and Reuse

- Cache only results whose context, policy, tenant, task, and model assumptions remain valid.
- Include tenant/scope and relevant version identifiers in cache keys.
- Invalidate AI outputs when material source facts, candidate versions, policies, or dependency assumptions change.
- Never use cached AI output as evidence that a new candidate is verified.
- Cache deterministic context summaries separately from model responses where useful.

23. Security and Authorization Integration

- AI requests inherit authorization from the initiating product action.
- The AI service must not broaden access beyond the caller's authorized tenant/scope.
- Context Builder performs authorization filtering before data reaches the model.
- AI results returned to users remain subject to normal object-level authorization.
- Promotion and cloud execution endpoints independently authorize actions; AI authorization is never sufficient.
- Administrative model/provider configuration changes require privileged access and audit.

24. Configuration and Governance

Configuration	Governance Requirement
Model allowlist	Only approved model/provider routes can run production tasks.
Prompt template	Versioned, reviewed, evaluated before material rollout.
Output schema	Versioned and backward-compatibility policy defined.
Task policy	Defines allowed inputs, output authority, tools, and risk class.
Tool registry	Every tool has scope, permission, risk, timeout, and audit metadata.
Evaluation threshold	Release gate for model/prompt changes.
Budget/quota	Per-tenant and global limits where appropriate.

25. AI Change Management

21. Create or modify a task, prompt, schema, model route, or tool configuration in version control.
22. Run unit and schema tests.
23. Run relevant golden, adversarial, security, and regression evaluations.
24. Compare against the current production baseline.
25. Review failures and material quality/safety regressions.
26. Approve configuration promotion according to engineering governance.
27. Roll out using controlled deployment and monitoring.
28. Retain the previous configuration for safe rollback.

26. MVP Architecture

The MVP should remain intentionally simple while preserving the architectural boundaries needed for future scale.

MVP Component	Implementation Role
AI Gateway/Orchestrator	Application module or small dedicated service.
Context Builder	Domain service using canonical relational/graph-derived queries.
Model Adapter	One hosted provider adapter behind stable interface.
Mock Adapter	Deterministic test adapter.
Prompt Registry	Version-controlled templates/configuration.
Output Validator	Schema + domain + policy validation.
Tool Broker	Read-only tools only if required; mutation excluded from MVP.
AI Trace Store	Relational metadata + controlled object artifacts.
Evaluation Runner	Batch test runner using golden/adversarial cases.

MVP Component	Implementation Role
Queue	Async AI jobs and retries.

This architecture deliberately avoids premature microservices. Components may begin as modules within the CloudShield AI application and workers, then be extracted when scale, isolation, or operational ownership justifies it.

27. Future Local Model Architecture

A local/open-weight model is a future deployment option, not an MVP prerequisite. The AI abstraction layer allows a LocalLLMAdapter to replace or complement the hosted adapter.

Capability	Hosted Model	Future Local Model
Provider interface	Common ModelRequest/Response contract	Same contract
Context controls	CloudShield-side	CloudShield-side
Output validation	CloudShield-side	CloudShield-side
Safety gates	CloudShield-side	CloudShield-side
DAIL authority	None	None
Model training	Provider-managed/pretrained	Optional future fine-tuning/domain adaptation
Deployment	External API	Private/self-hosted infrastructure

Moving to a local model must not weaken deterministic validation, authorization, evidence, or DAIL controls.

28. Security Acceptance Criteria

29. No model response can directly invoke unrestricted cloud APIs.
30. Every production AI task has a registered task definition and output schema.
31. AI context is filtered by tenant and authorization before model invocation.
32. Raw cloud credentials are excluded from AI prompts and model-visible tool results.
33. Malformed model output cannot create a trusted infrastructure state.
34. Infrastructure-affecting AI output becomes an explicit candidate before DAIL processing.
35. Prompt injection and indirect prompt injection test cases are included in release evaluation.
36. Material AI outputs can be traced to model/provider/version, prompt/schema version, and context references.
37. Model/provider failures produce explicit, observable failure states rather than fabricated results.
38. Model upgrades require regression and safety evaluation before production rollout.

29. Functional Acceptance Criteria

39. The system can explain a security finding using canonical finding and evidence context.
40. The system can produce a structured risk rationale with explicit uncertainties.
41. The system can generate a remediation recommendation tied to stable asset identifiers.
42. The system rejects a recommendation that references nonexistent or unauthorized resources.
43. The system can convert a valid infrastructure recommendation into a RemediationCandidate.
44. The candidate retains exact AI/context provenance required for downstream verification.
45. DAIL can consume the candidate without treating the AI response itself as verified.
46. Users can distinguish observed facts from AI inference and recommendation.

47. AI results remain useful when the model provider is unavailable by falling back to deterministic product functionality where possible.

30. Traceability

Specification	AI Architecture Dependency
00 — Master System Specification	Product architecture, intelligence role, governance, DAIL positioning.
01 — Software Requirements Specification	AI abstraction, structured outputs, safety, provider flexibility, MVP non-requirement for foundation-model training.
02 — System Architecture Specification	Intelligence service boundaries, data flow, async workers, provider adapters.
03 — Functional Specification	Finding explanation, risk, remediation, candidate, verification, promotion workflows.
04 — Product & UX Specification	AI explanation, recommendation, uncertainty, safe-path UX.
05 — Domain & Data Model	AIRequest, AIResponse, AIContext, AIRecommendation, Candidate, Evidence.
06 — State & Lifecycle	Candidate/version binding, invalidation, trusted-state rules.
07 — Identity & Dependency	Stable IDs, graph context, provenance, impact semantics.
08 — Security & Threat Model	Prompt injection, tool abuse, data isolation, secrets, AI trust boundary.

31. Engineering Decisions

Decision	Rationale
Use existing LLMs for MVP	Product value is contextual intelligence + deterministic safety, not foundation-model training.
Provider-neutral adapter	Avoid vendor lock-in and permit future local/private models.
Structured outputs	Enable deterministic parsing and validation.
No direct model-to-cloud authority	Reduces blast radius and preserves governance.
Canonical domain context	Improves grounding, traceability, and tenant isolation.
DAIL remains independent safety boundary	Prevents probabilistic reasoning from becoming lifecycle authority.
Evaluation as infrastructure	Makes model/prompt changes measurable and reversible.
Modular MVP	Reduces operational complexity while preserving clean future extraction boundaries.

32. Definition of Done for AI Architecture

- AI abstraction contract implemented and covered by tests.
- At least one approved hosted model adapter implemented.
- Mock adapter available for deterministic tests.
- Context Builder enforces tenant/scope authorization.
- Prompt/template registry is versioned.

- Structured output schemas are versioned.
- Output validation rejects invalid identifiers and unsupported changes.
- No raw credentials enter AI context.
- AI-generated infrastructure changes enter RemediationCandidate/DAIL.
- Golden and adversarial evaluation sets exist.
- AI telemetry captures safe metadata and validation outcomes.
- Provider failure and retry behavior is tested.
- Model/prompt change process is documented and gated.
- Traceability to product/domain/lifecycle/security specifications is established.

33. Glossary

Term	Definition
AI Abstraction Layer	Provider-neutral interface separating CloudShield AI from specific model providers.
AI Context	Authorized, curated domain information supplied to an AI task.
AI Orchestrator	Component coordinating task selection, context, model invocation, validation, and persistence.
Model Adapter	Provider-specific implementation of the common model interface.
Context Builder	Component that assembles relevant authorized domain facts.
Tool Broker	Security-controlled intermediary that evaluates model-requested tool actions.
Structured Output	Machine-readable AI response conforming to a registered schema.
Groundedness	Degree to which an AI claim is supported by supplied canonical evidence/context.
RemediationCandidate	Explicit proposed infrastructure change subject to DAIL lifecycle controls.
DAIL	Deterministic AI Infrastructure Lifecycle subsystem responsible for deterministic change lifecycle safety.
Trusted State	State earned after successful governed promotion; never directly produced by AI.